



Nädal 4

Nädal 4: SQL Agregatsioon

GROUP BY ja Agregaatfunktsioonid — Anna Metsa Väljakutse

Anna Mets, Marketing Lead

Sessiooni ülevaade (90 min)

Faas	Aeg	Sisu
CONNECTION	0:00–0:15	Anna ja Kristi dialoog, reaalmaailma analoogia
CONCEPTS	0:15–0:45	GROUP BY, agregaatfunktsioonid, HAVING, CTE, window functions
CONCRETE PRACTICE	0:45–1:20	Juhendatud pärimine, flash-harjutused, iseseisev praktika
CONCLUSIONS	1:20–1:30	Kokkuvõte, eelvaade Sessioon 2-le



CONNECTION

0:00–0:15

Anna Metsa juhatuse koosoleku väljakutse

Anna ja Kristi dialoog

Anna Mets, Marketing Lead

"Eelmisel nädalal Anna näitas TOP 20 toodete analüüsi Kristile. Kristi oli muljet avaldatud ja küsis kohe: 'See on suurepärane, aga ma tahan teada — kuidas meie müük muutub kuude lõikes? Mis on meie keskmine tellimusväärtus? Kes on meie parimad kliendid?'"



Seos olemasolevaga

- ◆ Kas te saaksite vastata neile küsimustele ainult SELECT ja JOIN-iga?
- ◆ Mida tähendab 'keskmine' või 'kokku võtta'?
- ◆ Millised äriküsimused on teile päriselus ette tulnud, mis nõuavad kokkuvõtet?
- ◆ Kujutage ette karpi täis müügikviitungeid. SELECT on nagu ühe kviitungi lugemine. JOIN on kahe erineva karbi võrdlemine. GROUP BY on nagu kviitungite sorteerimine kuhjadesse ja iga kuhi summa arvutamine.



CONCEPTS

0:15–0:45

GROUP BY, agregaatfunktsioonid, HAVING, CTE, window functions

GROUP BY loogika — 10,118 reast 26 reaks

```
-- Ilma GROUP BY-ta: 10,118 rida (iga tellimus eraldi)
SELECT order_date, total_amount FROM sales_orders;

-- GROUP BY-ga: 26 rida (iga kuu summa)
SELECT
    DATE_TRUNC('MONTH', order_date) AS kuu,
    SUM(total_amount) AS kogukäive
FROM sales_orders
WHERE order_date >= '2023-01-01'
GROUP BY DATE_TRUNC('MONTH', order_date)
ORDER BY kuu;
```

GROUP BY koondab read gruppidesse. 10,118 reast said 26 rida — see on agregatsiooni jõud!

SQL päringu täitmise järjekord

- ◆ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
- ◆ WHERE filtreerib ENNE grupeerimist (üksikread)
- ◆ HAVING filtreerib PÄRAST grupeerimist (kokkuvõtted)
- ◆ SELECT ja ORDER BY on viimased sammud

SQL taitmise järjekord (execution order)

- 1 FROM — millisest tabelist?
- 2 WHERE — milliseid RIDU jatta? (enne grupeerimist)
- 3 GROUP BY — kuidas grupeerida?
- 4 HAVING — milliseid GRUPPE jatta? (parast grupeerimist)
- 5 SELECT — mida näidata?
- 6 ORDER BY — kuidas sorteerida?
- 7 LIMIT — mitu rida?

NB! See erineb KIRJUTAMISE järjekorrast!



Agregaاتفunktsioonid

Funktsioon	Mida teeb	Näide
COUNT(*)	Loeb kõik read	Tellimuste arv
COUNT(DISTINCT x)	Loeb unikaalseid	Erinevate klientide arv
SUM(x)	Liidab kokku	Kogukäive
AVG(x)	Arvutab keskmise	Keskmine tellimus
MIN(x) / MAX(x)	Väikseim / suurim	Odavaim / kalleim toode



Tavaline viga — non-aggregated column

```
-- VALE! product_name pole agregeeritud ega GROUP BY-s
SELECT product_name, category, SUM(quantity) AS müüdnud
FROM products
GROUP BY category;
-- PostgreSQL annab vea!

-- ÕIGE! Lisa product_name GROUP BY-sse
-- või kasuta agregatsioonifunktsiooni
SELECT category, SUM(quantity) AS müüdnud
FROM products
GROUP BY category;
```

Reegel: iga veerg SELECT-is peab olema kas GROUP BY-s VÕI agregatsioonifunktsioon. PostgreSQL annab selge veateate.

WHERE vs HAVING

WHERE

- ◆ Filtreerib ENNE grupeerimist
- ◆ Töötab üksikriidadega
- ◆ Nagu turvamees uksele
- ◆ Näide: WHERE order_date >= '2024-01-01'

HAVING

- ◆ Filtreerib PÄRAST grupeerimist
- ◆ Töötab kokkuvõtetega
- ◆ Nagu kohtunik pärast mängu
- ◆ Näide: HAVING COUNT(*) > 100



WHERE ja HAVING koos — praktiline näide

```
-- WHERE: ainult 2024. aasta tellimused
SELECT city, COUNT(*) AS arv
FROM sales_orders
WHERE order_date >= '2024-01-01'
GROUP BY city;

-- HAVING: ainult linnad, kus rohkem kui 100 tellimust
SELECT city, COUNT(*) AS arv
FROM sales_orders
GROUP BY city
HAVING COUNT(*) > 100;
```

WHERE filtreerib enne grupeerimist (üksikread). HAVING filtreerib pärast grupeerimist (kokkuvõtted).

CTE (WITH klausel) — loetavuse võludus

```
WITH kuu_myyk AS (  
    SELECT  
        DATE_TRUNC('MONTH', order_date) AS kuu,  
        SUM(total_amount) AS käive  
    FROM sales_orders  
    WHERE order_date >= '2024-01-01'  
    GROUP BY DATE_TRUNC('MONTH', order_date)  
)  
SELECT kuu, käive,  
       LAG(käive) OVER (ORDER BY kuu) AS eelmine_kuu  
FROM kuu_myyk  
ORDER BY kuu;
```

CTE on nagu nimega vahetulem — loetavus paraneb drastiliselt. WITH annab alampäringule nime, mida saab hiljem kasutada.

Võtmeküsimused osalejatele

- ◆ Mis juhtub, kui lisate GROUP BY-sse veel ühe veeru?
- ◆ Mis on vahe COUNT(*) ja COUNT(customer_id) vahel?
- ◆ Millal kasutaksite CTE-d ja millal lihtsalt alampäringut?
- ◆ Miks on CTE loetavam kui pesastatud alampäring?



CONCRETE PRACTICE

0:45–1:20

Praktika — juhendatud näited ja iseseisev harjutamine

Näide 1: Müük kuude kaupa (mentor juhib)

```
SELECT
    TO_CHAR(order_date, 'YYYY-MM') AS kuu,
    COUNT(*) AS tellimusi,
    SUM(total_amount) AS käive,
    ROUND(AVG(total_amount), 2) AS keskmine_tellimus
FROM sales_orders
WHERE order_date >= '2024-01-01'
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY kuu;
```

Oodatav tulemus: 12 kuud, detsember kõrgeim (~175,390 EUR). TO_CHAR vormindab kuupäeva loetavaks.

Näide 2: TOP 10 klienti (mentor juhhib)

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name AS nimi,
  COUNT(o.order_id) AS tellimusi,
  SUM(o.total_amount) AS kogukäive,
  ROUND(AVG(o.total_amount), 2) AS keskmine
FROM customers c
JOIN sales_orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY kogukäive DESC
LIMIT 10;
```

JOIN + GROUP BY + ORDER BY + LIMIT koostöös. Kõik non-aggregated veerud on GROUP BY-s!

Näide 3: Tooted kategooriate kaupa (mentor juhib)

```
SELECT
    category,
    COUNT(*) AS tooteid,
    ROUND(AVG(price), 2) AS keskmine_hind,
    MIN(price) AS odavaim,
    MAX(price) AS kalleim
FROM products
WHERE is_active = TRUE
GROUP BY category
ORDER BY tooteid DESC;
```

Mitu agregaatfunktsiooni ühes päringus. WHERE filtreerib enne grupeerimist ainult aktiivsed tooted.

Iseseisev harjutus — kirjuta ise!

Ha-tase: sa tead struktuuri, aga valid ise filtrid ja grupeerimised.

```
-- 1. Müük linnade kaupa
SELECT c.city, COUNT(*) AS tellimusi,
       SUM(o.total_amount) AS kogukäive
FROM customers c
JOIN sales_orders o ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY kogukäive DESC;

-- 2. Tellimused päevade kaupa (viimased 30 päeva)
SELECT DATE_TRUNC('DAY', order_date) AS päev,
       COUNT(*) AS tellimusi
FROM sales_orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY DATE_TRUNC('DAY', order_date)
ORDER BY päev;
```

 Vihje: Kasuta JOIN, GROUP BY ja ORDER BY. HAVING lisab filtri kokkuvõtetele.



CONCLUSIONS

1:20–1:30

Kokkuvõte ja eelvaade Sessioon 2-le

Kokkuvõte — mida me täna õppisime?

- ◆ GROUP BY koondab read gruppidesse — üksikutest ridadest kokkuvõtted
- ◆ Agregaatfunktsioonid: COUNT, SUM, AVG, MIN, MAX
- ◆ WHERE filtreerib enne, HAVING filtreerib pärast grupeerimist
- ◆ CTE (WITH) teeb keerulised päringud loetavaks
- ◆ Te ei kirjutanud ainult SQL-i — te vastasite äriküsimustele!



Kokkuvõte — mida me täna õppisime? (jätkub)

- ◆ Sessioon 3 demo: grupi esitlus — iga liige räägib, 90 sek per inimene



Anna Metsa eelvaade Sessioon 2-le

Anna Mets, Marketing Lead

"Homme teeme grupitööd: iga meeskonnaliige saab oma andmedomeeni — müük, kliendid, inventuur, turundus. Eesmärk: koostada Kristile juhatuse koosoleku raport. Igaüks analüüsib oma valdkonda ja lõpuks paneme tulemused kokku!"

